Liquid-过滤器

> 过滤器是修改数字、字符串、变量和对象输出的简单方法。 它们被放置在一个输出标记{{}}中，并由一个管道字符|表示。可以在一个输出上使用多个过滤器。 它们是从左到右应用的。过滤器冒号后面接参数。

Plain Text

```
<!-- product.title = "Awesome Shoes" -->
{{ product.title | upcase | remove: "AWESOME"  }}
//在这个例子中，product是对象，title是属性，upcase、remove是过滤器
```

1. 数组过滤器

join

通过指定分隔符将数组所有元素拼接成一个字符串。

Plain Text

```
{{ product.tags | join: ', ' }}
```

first

返回数组第一个元素。

Plain Text

```
{{ product.tags | first }}
```

通过点符号获取数组第一个元素：

Plain Text

```
{% if product.tags.first == "sale" %}
  This product is on sale!
{% endif %}
```

last

返回数组最后一个元素。

Plain Text

```
{{ product.tags | last }}
```

通过点符号获取数组最后一个元素：

Plain Text

```
{% if product.tags.last == "sale"%}
  This product is on sale!
{% endif %}
```

在字符串中使用 last 过滤器，返回字符串的最有一个字符：

Plain Text

```
<!-- product.title = "Awesome Shoes" -->
{{ product.title | last }}
```

concat

合并两个或多个数组。

Plain Text

```
{% assign fruits = "apples, oranges, peaches, tomatoes" | split: ", " %}
{% assign vegetables = "broccoli, carrots, lettuce, tomatoes" | split: ", " %}
{% assign plants = fruits | concat: vegetables %}
{{ plants | join: ", " }}
```

index

返回数组中指定下标的元素，注意：并不是通过管道符的方式，而是通过中括号。

Plain Text

```
<!-- product.tags = "sale", "mens", "womens", "awesome" -->
{{ product.tags[2] }}
```

reverse

颠倒数组中元素的位置。

Plain Text

```
{% assign my_array = "apples, oranges, peaches, plums" | split: ", " %}
{{ my_array | reverse }}
```

size

返回数组元素个数，或字符串字符个数。

Plain Text

```
{{ 'The quick brown fox jumps over a lazy dog.' | size }}
```

通过点符号的方式：

Plain Text

```
{% if collections.frontpage.products.size > 10 %}
  There are more than 10 products in this collection!
{% endif %}
```

sort

按数组中元素的给定属性对数组中的元素进行排序。

Plain Text

```
{% assign products = collection.products | sort: 'price' %}
{% for product in products %}
  <h4>{{ product.title }}</h4>
{% endfor %}
```

where

对数组中的元素进行过滤筛选，类似于js的Array.prototype.filter，过滤器接收键（值为truthy），或键值，符合要求的将其过滤出来组成新的数组。

下例中过滤出类型为 kitchen 的产品：

Plain Text

```
{% assign kitchen_products = collection.products | where: "type", "kitchen" %}
```

下例中过滤出可用的产品：

Plain Text

```
{% assign available_products = collection.products | where: "available" %}
```

uniq

数组去重。

Plain Text

```
{% assign fruits = "orange apple banana apple orange" %}
{{ fruits | split: ' ' | uniq | join: ' ' }}
```

输出：

Plain Text

```
orange apple banana
```

2. HTML过滤器

HTML过滤器根据Liquid属性或商店资产创建HTML元素。

image_tag

生成img标签。您可以调整图像的宽高比或响应大小表现、预加载图像、添加alt文本或向标签传递其他HTML属性。

- image_url

使用 image_url 过滤器必须带有宽度或高度参数。

Plain Text

```
{{ settings.favicon | image_url: width: 2000 }}
```

输出：

Plain Text

```
//cdn.shopify.com/s/files/1/0593/8623/3013/files/32.png?v=1631784870&width=2000
```

- image_tag

生成一个img标签， image_tag 过滤器必须和 image_url 过滤器一起使用。

Plain Text

```
{{ settings.favicon | image_url: width: 2000 | image_tag }}
```

输出：

Plain Text

```
<img
src="//cdn.shopify.com/s/files/1/0593/8623/3013/files/32.png?v=1631784870&amp;w
"
  alt=""
srcset="//cdn.shopify.com/s/files/1/0593/8623/3013/files/32.png?v=1631784870&am
 352w,
//cdn.shopify.com/s/files/1/0593/8623/3013/files/32.png?v=1631784870&amp;width=
 832w,
//cdn.shopify.com/s/files/1/0593/8623/3013/files/32.png?v=1631784870&amp;width=
 1200w,
//cdn.shopify.com/s/files/1/0593/8623/3013/files/32.png?v=1631784870&amp;width=
 1920w"
  width="2000"
  height="2000">
```

- srcset

虽然 image_tag 会自动生成srcset，但您也可以指定自定义srcset关键字参数。srcset参数优先于宽度。但不建议设置自定义srcset，除非您希望通过将其设置为nil来完全删除该属性。

Plain Text

```
{{ settings.favicon | image_url: width: 2000 | image_tag: srcset: nil }}
```

输出：

Plain Text

```
<im
src="//cdn.shopify.com/s/files/1/0593/8623/3013/files/32.png?v=1631784870&amp;w
  alt=""
  width="2000"
  height="2000">
```

- 宽高属性

根据 image_url 过滤器中指定的尺寸和图像的宽高比自动添加到标签中。如果 image_tag 指定了任意一个宽高属性，则只使用所提供的属性。

Plain Text

```
{{ settings.favicon | image_url: width: 2000 | image_tag: srcset: nil, width: nil }}
{{ settings.favicon | image_url: width: 2000 | image_tag: srcset: nil, width: 200 }}
```

输出：

Plain Text

```
<img src="//cdn.shopify.com/s/files/1/0593/8623/3013/files/32.png?v=1631784870&amp;w" alt="">
<img src="//cdn.shopify.com/s/files/1/0593/8623/3013/files/32.png?v=1631784870&amp;w" alt="" width="200">
```

- sizes

可以使用 sizes 关键字参数指定源大小。此参数与 HTML sizes 属性相同的语法。

Plain Text

```
{{ settings.favicon | image_url: width: 200 | image_tag: sizes: '(min-width: 1100px) 535px, (min-width: 750px) calc((100vw - 130px) / 2), calc((100vw - 50px) / 2)' }}
```

输出：

Plain Text

```
<img
src="//cdn.shopify.com/s/files/1/0593/8623/3013/files/32.png?v=1631784870&amp;w"
  alt=""
  width="200"
  height="200"
  sizes="(min-width: 1100px) 535px, (min-width: 750px) calc((100vw - 130px) / 2), calc((100vw - 50px) / 2)">
```

- widths

虽然 image_tag 会自动生成srcset，但您可以使用 widths 关键字参数指定自己的 srcset 属性。此参数接受逗号分隔的整数宽度字符串。

Plain Text

```
{{ settings.favicon | image_url: width: 2000 | image_tag: widths: '200, 300, 400' }}
```

输出：

Plain Text

```
<img
src="//cdn.shopify.com/s/files/1/0593/8623/3013/files/32.png?v=1631784870&amp;w"
  alt=""
srcset="//cdn.shopify.com/s/files/1/0593/8623/3013/files/32.png?v=1631784870&am 200w,
//cdn.shopify.com/s/files/1/0593/8623/3013/files/32.png?v=1631784870&amp;width= 300w,
//cdn.shopify.com/s/files/1/0593/8623/3013/files/32.png?v=1631784870&amp;width= 400w"
  width="2000"
  height="2000">
```

- 额外的属性

提供给此过滤器的任何其他HTML属性也将添加到生成的HTML标签中。

Plain Text

```
{{ settings.favicon | image_url: width: 200 | image_tag: alt: 'My favicon', srcset: nil, class: 'css-class-1' }}
```

输出：

Plain Text

```
<img
src="//cdn.shopify.com/s/files/1/0593/8623/3013/files/32.png?v=1631784870&amp;w"
  alt="My favicon"
  width="200"
  height="200"
  class="css-class-1">
```

script_tag

生成一个 script 标签。

Plain Text

```
{{ 'shop.js' | asset_url | script_tag }}
```

输出:

Plain Text

```
<script
  src="//cdn.shopify.com/s/files/1/0593/8623/3013/t/207/assets/shop.js?75267"
  type="text/javascript"></script>
```

stylesheet_tag

生成一个 link 标签。

Plain Text

```
{{ 'shop.css' | asset_url | stylesheet_tag }}
```

输出：

```
Plain Text
<link
href="//cdn.shopify.com/s/files/1/0593/8623/3013/t/207/assets/shop.css?75268"
  rel="stylesheet"
  type="text/css"
  media="all" />
```

time_tag

将时间戳转换为 time 标签。

```
Plain Text
{{ article.published_at | time_tag }}
```

输出：

```
Plain Text
<time datetime="2021-12-17T08:19:19Z">Friday, December 17, 2021 at 4:19 pm +0800</time>
```

time_tag 过滤器接受与 Ruby 的 strftime 方法相同的参数。 你可以在 Ruby 的文档中找到一个简写格式的列表，或者使用像 strfti.me 这样的网站。 这些参数不会影响 <time> 标签中设置的 datetime 属性的值。

```
Plain Text
{{ article.published_at | time_tag: '%a, %b %d, %Y' }}
```

输出：

```
Plain Text
<time datetime="2021-12-17T08:19:19Z">Fri, Dec 17, 2021</time>
```

您可以向 time_tag 过滤器传递一个 format 参数，以输出店面语言的日期格式。 可用的格式有：

无法复制加载中的内容

自定义 datetime 属性格式：

Plain Text

```
{{ article.published_at | time_tag: '%a, %b %d, %Y', datetime: '%Y-%m-%d' }}
```

输出：

Plain Text

```
<time datetime="2021-12-17">Fri, Dec 17, 2021</time>
```

日期格式也可以在你的主题的区域设置中定义:

Plain Text

```
# /locales / en.json

"date_formats": {
  "month_day_year": "%B %d, %Y"
}
```

使用区域设置：

Plain Text

```
{{ article.published_at | time_tag: format: 'month_day_year' }}
```

输出：

Plain Text

```
<time datetime="2021-12-17T08:19:19Z">Dec 17, 2021</time>
```

payment_button

为托管产品的动态 checkout 按钮的父容器输出HTML。此筛选器必须用于在 product form 中的 form 对象。

Plain Text

```
{%- form 'product', product -%}
  {{ form | payment_button }}
{%- endform -%}
```

输出：

Plain Text

```
<form ...>
  ...
  <div data-shopify="payment-button" class="shopify-payment-button">
    ...
  </div>
  ...
</form>
```

注意：你不能通过AJAX请求呈现动态 checkout 按钮，包括那些通过Section Rendering API呈现的按钮。 动态签出按钮本身是由JavaScript添加的，由Shopify通过content_for_header对象添加的，该对象只在初始页面加载时运行。

payment_terms

注意，此过滤器只适用于已被邀请加入商店分期付款早期访问计划的合格商家。 如果你是Shopify商家，并没有被邀请提前访问，那么你可以加入候补名单。

呈现产品的Shop Pay Installments横幅。 此筛选器必须用于product form 中的 form 对象。

Plain Text

```
{%- form 'product', product -%}
  {{ form | payment_terms }}
{%- endform -%}
```

输出：

Plain Text

```
<form ...>
  ...
  <shopify-payment-terms variant-id="..." shopify-meta="...">
    ...
  </shopify-payment-terms>
  ...
</form>
```

payment_type_svg_tag

为内联目的返回请求的支付类型图像的 svg 标签。 与 shop.enabled_payment_types 变量一起使用。 接受一个可选的 class 属性，该属性应用于 svg 标签以控制图标的样式。

Plain Text

```
{% for type in shop.enabled_payment_types %}
  {{ type | payment_type_svg_tag, class: 'custom-class' }}
{% endfor %}
```

输出：

Plain Text

```
<svg class="custom-class" xmlns="http://www.w3.org/2000/svg">
  <circle fill="#EB001B" cx="15" cy="12" r="7"></circle>
  <circle fill="#F79E1B" cx="23" cy="12" r="7"></circle>
  ...
</svg>
<svg class="custom-class" xmlns="http://www.w3.org/2000/svg">
  <path fill="#494949" d="M9 11h20v2H9z"></path>
  ...
</svg>
```

preload_tag

生成带有preload rel属性的 link 标记。

Plain Text

```
{{ 'mobile_font.woff' | asset_url | preload_tag: as: 'font', media: 'mobile', type: 'font/woff' }}
```

输出：

Plain Text

```
<link
href="//cdn.shopify.com/s/files/1/0087/0462/t/394/assets/mobile_font.woff?28178"
  rel="preload"
  as="font"
  media="mobile"
  type="font/woff">
```

3. 数学过滤器

abs

返回数字的绝对值。

at_most

限定最大值。

at_least

限定最小值。

ceil

向上取整。

floor

向下取整。

minus

减。

plus

加。

times

乘。

divided_by

除。

round

四舍五入到指定位数小数，默认0。

Plain Text

```
{{ 4.6 | round }}
{{ 4.3 | round }}
{{ 4.5612 | round: 2 }}
```

输出：

Plain Text

```
5
4
4.56
```

modulo

取模。

4. money 过滤器

money

Plain Text

```
{{ 2000 | money }}
{{ 145 | money }}
```

输出：

Plain Text

```
$20.00
$1.45
```

money_with_currency

Plain Text

```
{{ 2000 | money_with_currency }}
{{ 145 | money_with_currency }}
```

输出：

Plain Text

```
$20.00 USD
$1.45 USD
```

money_without_trailing_zeros

Plain Text

```
{{ 2000 | money_without_trailing_zeros }}
{{ 145 | money_without_trailing_zeros }}
```

输出：

Plain Text

```
$20
$1.45
```

money_without_currency

Plain Text

```
{{ 2000 | money_without_currency }}
{{ 145 | money_without_currency }}
```

输出：

Plain Text

```
20.00
1.45
```

5. 字符串过滤器

append

往一个字符串后面附加另一个字符串。

Plain Text

```
{{ 'sales' | append: '.jpg' }}
```

输出：

Plain Text

```
sales.jpg
```

prepend

往字符串前面插入另一个字符串。

base64_encode

把字符串进行 base64 编码。

Plain Text

```
{{ 'one two three' | base64_encode }}
```

输出：

Plain Text

```
b25lIHR3byB0aHJlZQ==
```

base64_decode

把字符串进行 base64 解码。

Plain Text

```
{{ 'b25lIHR3byB0aHJlZQ==' | base64_decode }}
```

输出：

Plain Text

```
one two three
```

base64_url_safe_encode

将字符串编码为url安全的Base64。 为了生成url安全的Base64，这个过滤器使用-和_来代替+和_。

Plain Text

```
{{ '<p>ok?</p>' | base64_url_safe_encode }}
```

输出：

Plain Text

```
PHA-b2s_PC9wPg==
```

base64_url_safe_decode

从url安全的Base64解码字符串。

Plain Text

```
{{ 'PHA-b2s_PC9wPg==' | base64_url_safe_decode }}
```

输出：

Plain Text

```
<p>ok?</p>
```

camelcase

大驼峰。

capitalize

首单词首字母大写。

downcase

小写。

upcase

大写。

escape

转义HTML标签。

handle/handleize

将字符串格式化为handle。

符串被转换为小写，任何空格和特殊字符被连字符(-)替换。 如果标题包含多个连续的空格或特殊字符，则用一个连字符替换它们。 句柄的开头和结尾去掉空格和特殊字符。

md5

MD5加密。

sha1

sha1加密。

sha256

sha256加密。

hmac_sha1

hmac_sha1加密。

Plain Text

```
{% assign my_secret_string = 'ShopifyIsAwesome!' | hmac_sha1: 'secret_key' %}
My encoded string is: {{ my_secret_string }}
```

hmac_sha256

hmac_sha256加密。

Plain Text

```
{% assign my_secret_string = 'ShopifyIsAwesome!' | hmac_sha256: 'secret_key' %}
My encoded string is: {{ my_secret_string }}
```

newline_to_br

在字符串的每个换行符前插入
换行HTML标记。

pluralize

根据数字的值输出英文字符串的单数或复数版本。 第一个参数是单数字符串，第二个参数是复数字符串。

Plain Text

```
{{ cart.item_count }}
{{ cart.item_count | pluralize: 'item', 'items' }}
```

输出：

Plain Text

```
3 items
```

remove

从字符串中删除所有出现的子字符串。

Plain Text

```
{{ 'Hello, world. Goodbye, world.' | remove: 'world' }}
```

输出：

Plain Text

```
Hello, . Goodbye, .
```

remove_first

只从字符串中移除第一个出现的子字符串。

replace

用子字符串替换所有出现的字符串。

Plain Text

```
{{ 'Hello, world. Goodbye, world.' | replace: 'world', 'Jack' }}
```

输出：

Plain Text

```
Hello, Jack. Goodbye, Jack.
```

replace_first

将第一个出现的字符串替换为子字符串。

slice

从指定下标开始截取指定长度的子字符串，如果下标为负数，则从末尾开始算起，如果不传递长度，则长度为1。

split

使用指定的分隔符将字符串分割成字符串数组。

strip

删除字符串两边的空白符（空格、制表符、换行符、换页符）

lstrip

删除字符串左边的空白符。

rstrip

删除字符串右边的空白符。

strip_html

删除字符串中所有的html标签。

strip_newlines

从字符串中删除所有换行符。

truncate

截断字符串到指定字符数并添加省略号（省略号也算在字符数里）。

可传递第二个参数替代三个点的省略号字符，甚至传递空字符串来去除省略号。

Plain Text

```
{{ 'The cat came back the very next day' | truncate: 13 }}
{{ 'The cat came back the very next day' | truncate: 13, ', and so on' }}
{{ 'The cat came back the very next day' | truncate: 13, '' }}
```

输出：

Plain Text

```
The cat ca...
Th, and so on
The cat came
```

truncatewords

截断字符串到指定单词数并添加省略号（省略号**不算**在单词数里）。

可以传递第二个参数替代三个点的省略号字符，甚至传递空字符串来去除省略号。

Plain Text

```
{{ 'The cat came back the very next day' | truncatewords: 4 }}
{{ 'The cat came back the very next day' | truncatewords: 4, '--' }}
{{ 'The cat came back the very next day' | truncatewords: 4, '' }}
```

输出：

Plain Text

```
The cat came back...
The cat came back--
The cat came back
```

url_encode

将字符串中的任何url不安全字符转换为百分比编码的字符。

注意，空格会转换为+号，而不是百分比编码的字符。

url_escape

标识字符串中不允许出现在url中的所有字符，并将这些字符替换为它们的转义变体。

类似于js中的encodeURI()。

url_param_escape

用转义的变量替换url中不允许的字符串中的所有字符，包括&。